22MDC56 – MACHINE LEARNING LABORATORY

PROJECT REPORT

Bank IQ: A Machine Learning Intelligence Layer for Bank-Statement Analytics — Transaction Classification, Customer Segmentation and Spend Forecasting on a Regulatory Digital Twin

Domain: Machine Learning – Banking / Financial Services Analytics

Submitted by

SANDHRU D.

Register No.: _____

Under the guidance of

_____

(Project Guide)

M.Sc. Decision and Computing Sciences – V Semester

Coimbatore Institute of Technology, Coimbatore

Academic Year 2026–27

BONAFIDE CERTIFICATE

Certified that this project report titled “Bank IQ: A Machine Learning Intelligence Layer for Bank-Statement Analytics — Transaction Classification, Customer Segmentation and Spend Forecasting on a Regulatory Digital Twin” is the bonafide work of SANDHRU D. (Register No. _____) who carried out the project work under my supervision during the academic year 2026–27 for the course 22MDC56 Machine Learning Laboratory, M.Sc. Decision and Computing Sciences, Coimbatore Institute of Technology.

_____ _____

Project Guide Head of the Department

Submitted for the project viva-voce examination held on _____.

_____ _____

Internal Examiner External Examiner

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, _____, for the guidance, review and encouragement throughout this work, and to the Head of the Department and the faculty of the Department of Computing, Coimbatore Institute of Technology, for the laboratory facilities and the course structure that made an end-to-end project possible. I thank my Vasool Raja team-mates — Nandana R, Naveen Kumar D and Nethra Devi R — on whose hackathon work the rule engine and application that this project extends were built, and my family and friends for their support.

Sandhru D.

Abstract

A bank statement records everything that happens to an account but explains nothing: which lines are charges, where the money goes, whether a debit is unusual, what next month will cost. Existing tools, including the author's own hackathon project Vasool Raja — a Regulatory Digital Twin that runs 19 RBI customer-protection rules over a parsed statement — classify transactions with hand-written regular expressions, which are exact on familiar formats and silent on the long tail of abbreviations, bank-specific codes and OCR noise. Bank IQ keeps that rule engine unchanged and adds a classical machine-learning intelligence layer trained on a labelled synthetic dataset of 1,27,604 Indian bank-statement lines (500 customers × 12 months, 10 categories, 5 hidden customer archetypes, seasonality and noisy narrations).

Every line is represented by TF-IDF word and character n-grams plus nine numeric features. Under stratified 5-fold cross-validation the regex baseline reaches 71.5% accuracy (macro-F1 0.794) on the held-out test split; Complement Naive Bayes 93.4%, kNN 97.4%, a Decision Tree 96.7% and Logistic Regression 99.9% (macro-F1 0.997); Random Forest reaches 99.7% and bagging, AdaBoost and gradient boosting are compared against the single tree. Next-month spend is predicted from a 34-feature customer-month panel under GroupKFold-by-customer cross-validation: Gradient Boosting (400) reaches test R^2 0.871 and RMSE ₹11,881 against 0.832 and ₹13,590 for the naive “same as this month” baseline. K-Means and Ward clustering on a 13-feature behavioural profile choose k = 5 by silhouette and recover the hidden customer archetypes with Adjusted Rand Index 0.93; PCA needs 9 of 13 components for 95% variance; a from-scratch locally weighted regression of spend on income is studied across bandwidths. The layer is deployed as FastAPI endpoints and an Intelligence dashboard inside the existing Tamil/English application — ML categories beside rule categories, segment, forecast, explainable anomaly flags and plain-language business insights — with 13 new tests, seven notebooks and a Docker build.

Keywords: Transaction Classification, TF-IDF, Naive Bayes, Decision Tree, kNN, Logistic Regression, Bagging, Random Forest, AdaBoost, Gradient Boosting, Regression, Cross-Validation, K-Means, Hierarchical Clustering, PCA, Locally Weighted Regression, Anomaly Detection, FastAPI

Table of Contents

(Right-click → Update Field in Word to refresh page numbers.)

1. Introduction

1.1 Background

Every bank account produces a stream of transaction lines, each described by a short cryptic narration. Personal-finance apps, bank analytics teams and consumer-protection tools all need the same first step — turn each line into a category — and then the same next steps: aggregate categories into monthly behaviour, compare the customer with others, anticipate the next month, and notice what is unusual. Vasool Raja does the consumer-protection part deterministically: it parses a statement, rebuilds the account as a digital twin and evaluates RBI rules to produce findings with evidence chains. Its transaction classifier is a set of regular expressions.

1.2 Motivation

Regular expressions cannot learn. On the narration formats they were written for they are perfect; on everything else — merchant-only lines, abbreviations, OCR output from a passbook — they are silent or wrong. Machine learning replaces the string with the pattern: a classifier trained on labelled examples generalises to formats it has never seen. Once lines are categorised reliably, the same account history supports customer-level analytics the rule engine never had: a behavioural segment, a spend forecast, an anomaly flag and a set of insights a person can act on. The project applies the complete classical machine-learning syllabus to one real product, with the existing regex classifier as the baseline every model must beat.

1.3 Problem Statement

Given the transaction lines of a bank account (date, narration, debit, credit, balance) as produced by the Vasool Raja parsers, build, compare and deploy classical machine-learning models that classify each line into one of ten transaction categories more accurately than the rule-based classifier; predict a customer's next-month spend; segment customers by behaviour without demographic data; flag unusually large debits in an explainable way; and present the outputs as business insights on a dashboard integrated with the existing application, with the rule engine remaining the only component that files a claim.

1.4 Objectives

1. A labelled, reproducible dataset of Indian bank-statement lines and its EDA.
2. A preprocessing pipeline: narration cleaning, TF-IDF word/char n-grams, numeric features, customer-month panel, customer behavioural profile.
3. Transaction classification with Decision Tree, Naive Bayes, kNN and Logistic Regression under stratified 5-fold CV against the regex baseline; ensembles (Bagging, Random Forest, AdaBoost, Gradient Boosting).
4. Next-month spend regression with Linear, Ridge, Lasso, polynomial, tree, forest and boosting models under GroupKFold cross-validation against a naive baseline.
5. Customer segmentation with K-Means and Ward clustering, validated by silhouette and ARI; PCA and Truncated SVD studies; locally weighted regression from scratch.
6. An intelligence service and API, an Intelligence dashboard in the existing bilingual app, tests, notebooks and a Docker build.

1.5 Scope

In scope: the ten-category taxonomy, single-account analytics, the classical models above and the dashboard. Out of scope: deep learning and large language models, labelled fraud detection, real customer data (models are validated on the synthetic test split and demonstrated on the app's synthetic sample statements), and any change to the rule engine.

1.6 Organisation of the Report

Chapter 2 surveys the literature; Chapters 3–5 cover analysis, design and implementation; Chapter 6 the dataset, preprocessing and EDA; Chapter 7 model building; Chapter 8 results per syllabus concept; Chapter 9 business insights; Chapter 10 deployment and demonstration; Chapter 11 concludes.

2. Literature Survey

Sixteen references were reviewed in the order the techniques appear in the pipeline: text representation, single-model learners, ensembles, non-parametric regression, clustering and validation, dimensionality reduction, evaluation protocol and financial anomaly detection.

2.1 Induction of Decision Trees

J. R. Quinlan, *Machine Learning*, *vol. 1*, *no. 1*, *pp. 81–106*, *1986*.

Summary: Introduces ID3, which grows a decision tree top-down by choosing at each node the attribute with the highest information gain, and discusses noise, missing values and the over-fitting of fully grown trees.

Relevance: The Decision Tree classifier for transaction categories and the regression tree for next-month spend follow this greedy impurity-driven construction (CART in scikit-learn); the depth sweep in the results reproduces the over-fitting the paper describes.

Limitation: A single tree is unstable — small changes in the data change the tree — which motivates the ensemble methods.

2.2 Nearest Neighbor Pattern Classification

T. Cover and P. Hart, *IEEE Transactions on Information Theory*, *vol. 13*, *no. 1*, *pp. 21–27*, *1967*.

Summary: Proves that the nearest-neighbour rule's error is bounded by twice the Bayes error as the sample grows, establishing kNN as a non-parametric classifier with a strong theoretical guarantee.

Relevance: Justifies kNN as a baseline for narration classification; because it is distance based it runs on the dense SVD-200 projection of the TF-IDF matrix with cosine distance, and the k sweep shows the bias–variance effect of the neighbourhood size.

Limitation: No model is learned, so prediction is O(n) and the method is sensitive to feature scaling, sparsity and irrelevant features.

2.3 Bagging Predictors

L. Breiman, *Machine Learning*, *vol. 24*, *no. 2*, *pp. 123–140*, *1996*.

Summary: Shows that training many versions of an unstable learner on bootstrap resamples and aggregating them (voting or averaging) substantially reduces variance and improves accuracy, especially for decision trees.

Relevance: The Bagging (50 trees) classifier is exactly this recipe and is the first ensemble step above the single tree in the ensemble study.

Limitation: Gains are small for stable learners, and bagged trees remain correlated when a few strong features dominate.

2.4 Random Forests

L. Breiman, *Machine Learning*, *vol. 45*, *no. 1*, *pp. 5–32*, *2001*.

Summary: Extends bagging by also randomising the features considered at each split, which de-correlates the trees; introduces out-of-bag error estimates and variable-importance measures and shows strong results across many datasets.

Relevance: Random Forest is used for transaction classification (on the sparse TF-IDF matrix) and for spend regression; its n-gram importances explain which words the model relies on.

Limitation: Less interpretable than one tree; importance measures are biased toward high-cardinality features.

2.5 A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting

Y. Freund and R. E. Schapire, *Journal of Computer and System Sciences*, *vol. 55*, *no. 1*, *pp. 119–139*, *1997*.

Summary: Introduces AdaBoost, which trains weak learners sequentially, re-weighting misclassified examples so that later learners focus on the hard cases, and proves bounds on training and generalisation error.

Relevance: AdaBoost with depth-2 trees is one of the four ensembles compared; its weaker result on the 10-class narration problem illustrates its sensitivity to noisy, multi-class data.

Limitation: Sensitive to label noise and outliers because misclassified points receive ever-larger weights; slow with many classes.

2.6 Greedy Function Approximation: A Gradient Boosting Machine

J. H. Friedman, *The Annals of Statistics*, *vol. 29*, *no. 5*, *pp. 1189–1232*, *2001*.

Summary: Frames boosting as gradient descent in function space: each new tree fits the negative gradient of the loss, with shrinkage and subsampling for regularisation, giving a general method for regression and classification.

Relevance: Gradient Boosting is the best next-month spend regressor and is the model deployed for the forecast; HistGradientBoosting is one of the classification ensembles.

Limitation: Sequential training is slower than bagging, and depth, learning rate and number of trees interact.

2.7 Robust Locally Weighted Regression and Smoothing Scatterplots

W. S. Cleveland, *Journal of the American Statistical Association*, *vol. 74*, *no. 368*, *pp. 829–836*, *1979*.

Summary: Proposes fitting a low-degree polynomial at each query point by weighted least squares, with weights that decay with distance, and a robustness iteration that down-weights outliers; the bandwidth controls smoothness.

Relevance: The from-scratch Locally Weighted Regression of monthly spend on monthly income implements this idea with a Gaussian kernel and studies the bandwidth τ.

Limitation: Every prediction re-fits a local model (O(n) per query) and the method does not scale to many input dimensions.

2.8 Some Methods for Classification and Analysis of Multivariate Observations

J. MacQueen, *Proc. 5th Berkeley Symposium on Mathematical Statistics and Probability, vol. 1, pp. 281–297, 1967.*

Summary: Introduces the k-means procedure that assigns points to the nearest of k centroids and recomputes the centroids until convergence, minimising within-cluster variance.

Relevance: K-Means is the primary customer-segmentation method on the 14-feature behavioural profile; the elbow and silhouette decide k and the clusters are named as business segments.

Limitation: Requires k in advance, assumes roughly spherical clusters and is sensitive to initialisation and scaling.

2.9 Hierarchical Grouping to Optimize an Objective Function

J. H. Ward Jr., *Journal of the American Statistical Association, vol. 58, no. 301, pp. 236–244, 1963.*

Summary: Describes agglomerative clustering in which the two groups whose merger least increases the total within-group sum of squares are joined at each step, producing a full hierarchy (dendrogram).

Relevance: Ward clustering is compared with K-Means on the same customer profiles; the dendrogram gives a second view of the number of segments and the Adjusted Rand Index measures agreement.

Limitation: $O(n^2)$ memory; merges cannot be undone.

2.10 Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis

P. J. Rousseeuw, *Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987.*

Summary: Defines the silhouette of a point as (b − a)/max(a, b), where a is its mean distance to its own cluster and b to the nearest other cluster, giving a scale-free score in [−1, 1] for comparing clusterings.

Relevance: The silhouette score is the criterion used to choose k for both K-Means and Ward clustering and to compare the two algorithms.

Limitation: $O(n^2)$ pairwise distances; favours compact, well-separated clusters.

2.11 Principal Component Analysis: A Review and Recent Developments

I. T. Jolliffe and J. Cadima, *Philosophical Transactions of the Royal Society A, vol. 374, no. 2065, 20150202, 2016.*

Summary: Reviews PCA as the orthogonal projection maximising retained variance, its computation by eigen-decomposition or SVD, the choice of the number of components, and extensions such as sparse and robust PCA.

Relevance: PCA is applied to the standardised customer profile (variance, loadings, segment recovery vs components) and Truncated SVD plays the same role for the sparse TF-IDF matrix of narrations.

Limitation: Components are linear combinations of all features and can be hard to interpret; variance retained does not guarantee predictive information retained.

2.12 A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection

R. Kohavi, *Proc. 14th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1137–1143, 1995.*

Summary: Compares hold-out, k-fold, leave-one-out and bootstrap estimators of accuracy on real datasets and recommends stratified k-fold cross-validation as the best trade-off between bias and variance for model selection.

Relevance: All classifier comparisons use stratified 5-fold cross-validation on the training split with a separate held-out test set; the regression uses GroupKFold by customer so that no customer appears in both train and validation folds.

Limitation: Cross-validation estimates have variance themselves and k-fold with small k can be pessimistic.

2.13 Term-Weighting Approaches in Automatic Text Retrieval

G. Salton and C. Buckley, *Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.*

Summary: Compares term-frequency, inverse-document-frequency and length-normalisation schemes for representing text as weighted term vectors and shows that tf·idf with cosine normalisation gives the best retrieval performance.

Relevance: Every bank-statement narration is turned into a TF-IDF vector (word 1–2-grams and character 3–5-grams) before classification; the weighting lets rare, informative tokens (SALARY, NACH, REV) outweigh frequent reference tokens.

Limitation: Bag-of-words ignores word order and semantics; vocabulary grows with the corpus.

2.14 Tackling the Poor Assumptions of Naive Bayes Text Classifiers

J. D. M. Rennie, L. Shih, J. Teevan and D. R. Karger, *Proc. 20th International Conference on Machine Learning (ICML), pp. 616–623, 2003.*

Summary: Identifies why multinomial Naive Bayes under-performs on text (skewed class sizes, weight magnitude errors) and proposes Complement Naive Bayes with TF-IDF transforms and weight normalisation, which closes most of the gap to SVMs.

Relevance: Complement Naive Bayes is the Naive Bayes variant used for the 10-class narration problem, because class sizes are imbalanced and the features are TF-IDF counts rather than Gaussian measurements.

Limitation: Still assumes conditional independence of terms; calibration of probabilities is poor.

2.15 N-Gram-Based Text Categorization

W. B. Cavnar and J. M. Trenkle, *Proc. 3rd Annual Symposium on Document Analysis and Information Retrieval (SDAIR), pp. 161–175, 1994.*

Summary: Represents documents by their character n-gram frequency profiles and shows that such profiles are robust to spelling errors, truncation and OCR noise, giving reliable language and topic categorisation.

Relevance: Bank narrations are abbreviated, truncated and noisy (CHRG, WDL, RVSL, OCR from passbooks); character 3–5-grams are what let the classifier recover meaning that word tokens and hand-written regexes miss.

Limitation: Large, redundant feature spaces; n-grams carry no semantics beyond surface form.

2.16 A Survey of Anomaly Detection Techniques in Financial Domain

M. Ahmed, A. N. Mahmood and M. R. Islam, *Future Generation Computer Systems, vol. 55, pp. 278–288, 2016.*

Summary: Surveys statistical, clustering-based and classification-based anomaly-detection methods applied to financial data, with the taxonomy of point, contextual and collective anomalies and a discussion of the scarcity of labelled data.

Relevance: Bank IQ's unusual-debit flag is a contextual point anomaly: a robust z-score of the amount within its ML category. The survey justifies a transparent statistical detector where labelled fraud data is unavailable.

Limitation: Survey only; no benchmark on bank-statement data; most methods need tuning per dataset.

2.17 Comparative Analysis and Research Gap

No.	Paper (Year)	Technique	Strength	Limitation
1	Induction of Decision Trees (1986)	Decision tree induction (ID3)	Interpretable rules; handles mixed features	Unstable; over-fits without pruning
2	Nearest Neighbor Pattern Classification (1967)	k-nearest neighbours	Simple, non-parametric, strong asymptotic bound	Slow at prediction; weak on sparse text
3	Bagging Predictors (1996)	Bootstrap aggregating	Variance reduction for unstable learners	Trees stay correlated
4	Random Forests (2001)	Random forest	Robust, de-correlated trees, built-in importance	Black-box; importance bias
5	A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting (1997)	AdaBoost	Sequential focus on hard examples	Sensitive to noise; slow multi-class
6	Greedy Function Approximation (2001)	Gradient boosting	State-of-the-art on tabular data	Slower; several interacting hyperparameters
7	Robust Locally Weighted Regression and Smoothing Scatterplots (1979)	Locally weighted regression (LOWESS)	Flexible non-parametric fit	O(n) per prediction; low-dimensional
8	Some Methods for Classification and Analysis of Multivariate Observations (1967)	k-means clustering	Fast, scalable segmentation	k chosen by hand; spherical clusters
9	Hierarchical Grouping to Optimize an Objective Function (1963)	Agglomerative (Ward) clustering	No k needed up front; full hierarchy	$O(n^2)$ memory; greedy merges
10	Silhouettes (1987)	Silhouette score	Scale-free cluster quality	Favours compact clusters

No.	Paper (Year)	Technique	Strength	Limitation
			score	
11	Principal Component Analysis (2016)	PCA / SVD	Compact, decorrelated representation	Harder to interpret
12	A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection (1995)	Cross-validation	Reliable model selection protocol	Estimates still have variance
13	Term-Weighting Approaches in Automatic Text Retrieval (1988)	TF-IDF term weighting	Simple, strong text representation	No word order or semantics
14	Tackling the Poor Assumptions of Naive Bayes Text Classifiers (2003)	Complement Naive Bayes	Fast NB that handles class imbalance	Independence assumption; poor calibration
15	N-Gram-Based Text Categorization (1994)	Character n-gram features	Robust to noise, truncation, OCR	Large feature space
16	A Survey of Anomaly Detection Techniques in Financial Domain (2016)	Anomaly detection in finance	Taxonomy and methods for financial anomalies	No ready benchmark; per-dataset tuning

Table 2.1: Comparison of surveyed literature

The surveyed methods are mature individually; none is applied together to bank-statement data against the rule systems banks actually run, none builds classification, segmentation, forecasting and anomaly flags on one pipeline, none is deployed beside a consumer-protection rule engine, and no public labelled statement dataset exists. Bank IQ addresses each: a documented labelled generator, a side-by-side evaluation against the regex baseline with per-class gains, customer-level analytics with grouped cross-validation, and a deployed dashboard with a clear rule that models inform and rules decide.

3. System Analysis

3.1 Existing System

Vasool Raja's classify.py assigns a kind (CHARGE, ATM, REVERSAL, INTEREST, DEBIT, CREDIT …) and a channel (UPI, POS, ATM, NEFT …) to each transaction with regular expressions, then runs the rulebook. Mapped to the ten ML categories (bankiq/rulemap.py), it reaches 71.5% accuracy on the labelled test set. It cannot improve from examples, has no notion of a customer's behaviour, trend or expected spend, and is not evaluated.

- Silent on the long tail: merchant-only lines, abbreviations and OCR noise fall to UNKNOWN or the wrong class.
- No customer-level analytics: no segment, forecast or anomaly flag.
- Every new bank format needs a new regex; no measured accuracy.

3.2 Proposed System

Bank IQ adds a package bankiq/ beside the untouched engine: a labelled dataset generator, a feature module, six experiment scripts, an intelligence service, an API router and an Intelligence screen. The engine's parsers feed both the rulebook and the ML layer; the dashboard shows the two classifications side by side and never lets the model file a claim.

3.3 Feasibility

Technical: scikit-learn covers every model; the full experiment suite runs in about 25 minutes on two CPU cores and the service answers a 300-line statement in under a second. Operational: the layer is pure Python with no external service; models are joblib files loaded once. Economic: open-source stack, laptop hardware.

3.4 Requirements

Type	Requirement
Functional	Generate dataset; featurise; train and evaluate classifiers, ensembles, regressors, clusterers, PCA, LWR; analyse a statement (file or stored account) → categories, panel, segment, forecast, anomalies, insights; dashboard; model card
Non-functional	Reproducible (fixed seeds, one command); honest evaluation (held-out test, grouped CV, hidden archetype for clustering); explainable outputs; Tamil/English UI; no change to the rule engine
Software	Python 3.10+, scikit-learn 1.4+, pandas, NumPy, SciPy, matplotlib, joblib, FastAPI, pytest, Jupyter, Docker
Hardware	Any laptop; 2 cores, 4 GB RAM

Table 3.1: Requirements

4. System Design

4.1 Architecture

```
Statement (CSV / PDF / passbook photo)
  └─ vasool.parsers ──► transaction lines (date, narration, debit, credit, balance)
       ├─ RULE ENGINE (unchanged): classify (regex) → twin → 19 RBI rules → findings →
evidence pack → guardian → case
       └─ INTELLIGENCE LAYER (bankiq/):
            features.py   clean narration → TF-IDF word(1-2) + char_wb(3-5) + 9 numeric
features
                          month_panel (customer-month, 34 features) · customer_profile
(14 features)
            service.py    ML category (+ rule category) → category mix → monthly panel →
K-Means segment
                          → next-month forecast (Gradient Boosting) → anomaly z-scores →
insights
            api/ml.py     GET /api/ml/status · /results · /figures/{png}   POST
/api/ml/analyze · /analyze-json
                          GET /api/accounts/{id}/intelligence (session-protected)
            web           #/intelligence dashboard (vanilla JS, Tamil / English)
```

Figure 4.1: Architecture — the rule engine and the intelligence layer share the parsers

4.2 Module Design

Module	Responsibility	Key functions
bankiq/data/generator.py	Labelled synthetic dataset	generate(n_customers, months), load(); templates T (regular) and H (long tail); ARCHETYPES; SEASON
bankiq/features.py	All features	clean_narration, numeric_features, TransactionFeaturizer (fit/transform → sparse), month_panel, customer_profile

Module	Responsibility	Key functions
bankiq/rulemap.py	Regex baseline	rule_category(narration, debit, credit) — Vasool Raja Kind/Channel → ten categories
bankiq/experiments/*	One script per concept	eda, classification, ensembles, regression, clustering, pca_lwr, run_all → results/*.json, figures, models
bankiq/models_lwr.py	From scratch	LocallyWeightedRegression(tau).fit/predict, rmse
bankiq/service.py	Inference	models() (lazy joblib), classify (+ direction mask), analyze(rows), results_summary()
bankiq/api/ml.py	HTTP	router (/api/ml, public) and account_router (/api/accounts/{id}/intelligence, session)
web/static/app.js	UI	screenIntel / bindIntel / renderIntel / renderModelCard; i18n keys intel.*

Table 4.1: Modules

4.3 Data Design

transactions.csv: customer_id, date, narration, debit, credit, balance, bank, category, archetype. customers.csv: customer_id, bank, archetype, monthly_income, opening_balance, has_emi. The customer-month panel adds spend, prev_spend, income, credits, n_txn, n_debit, avg_debit, max_debit, end/min balance, month_idx, next_month (+sin/cos), amt_<category> × 10, n_<category> × 10 and the target next_spend. The customer profile holds monthly_spend, monthly_income, txn_per_month, avg_txn, upi/card/atm/transfer share, charges and refunds per month, has_emi, avg and min balance, spend_income_ratio.

4.4 API Design

Endpoint	Auth	Returns
GET /api/ml/status	public	available flag, model files
GET /api/ml/results	public	every results/*.json + figure list (model card)
GET /api/ml/figures/{name}.png	public	experiment figure
POST /api/ml/analyze (multipart file)	public	parse with vasool.parsers → full analysis
POST /api/ml/analyze-json {rows}	public	analysis of already-parsed rows
GET /api/accounts/{id}/intelligence	session; owner only	analysis of the stored account history

Table 4.2: Endpoints

4.5 User Interface Design

The Intelligence screen follows the app's existing screen/bind pattern and i18n dictionary. Top to bottom: source selector (my account / upload / sample buttons); five KPI tiles (transactions and period, total debits and income, charges and refunds, segment, forecast vs naive); business insights; category bars with ML labels; an inline-SVG monthly income-vs-spend chart; the behavioural profile table; the unusual-debits table; every line with its ML category, confidence and the rule's category, with a “show disagreements only” filter; and a collapsible model card that renders the experiment tables and figures from /api/ml/results.

5. Implementation

5.1 Technology Stack

Layer	Technology	Role
Language	Python 3.11, JavaScript (no build step)	Backend and UI
Data	pandas, NumPy	Generator, panel, profile
Text	TfidfVectorizer (word 1–2, token [A-Z0-9]+; char_wb 3–5), TruncatedSVD	Narration features
Models	scikit-learn 1.4+: LogisticRegression, ComplementNB, DecisionTree*, KNeighbors*, Bagging, RandomForest*, AdaBoost, HistGradientBoosting, Linear/Ridge/Lasso, PolynomialFeatures, GradientBoostingRegressor, KMeans, AgglomerativeClustering, PCA	All syllabus models
From scratch	NumPy	Locally weighted regression
Evaluation	StratifiedKFold, GroupKFold, metrics, SciPy dendrogram, matplotlib	Metrics and 15 figures
Serving	joblib, FastAPI, Uvicorn	Models, API
Engineering	pytest, Docker, Makefile, Jupyter	Tests, deployment, notebooks

Table 5.1: Technology stack

5.2 Project Structure

```
bankiq/            data/generator.py · features.py · rulemap.py · models_lwr.py ·
service.py · api/ml.py
                   experiments/ eda · classification · ensembles · regression ·
clustering · pca_lwr · run_all
notebooks/         01_eda … 07_service_demo      results/ *.json + figures/fig1–15
models/ *.joblib
tests/test_intelligence.py (13 tests)      docs/ (this report, Phase I, literature
summary, slides, viva)
vasool/ api/main.py web/   the Vasool Raja engine, API and UI (unchanged except the
mounted router and the new screen)
```

5.3 Key Code

Narration cleaning and features (bankiq/features.py)

```
s = _LD.sub(" ", s.upper())           # JAN26 -> JAN 26 ; 4123XXXX7710 -> 4123 XXXX 7710
s = _NUM.sub(" NUM ", s)              # any 2+ digit run -> one token: reference numbers
carry no meaning
self.word = TfidfVectorizer(ngram_range=(1, 2), token_pattern=r"[A-Z0-9]+", min_df=2,
sublinear_tf=True)
self.char = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5), min_df=3,
sublinear_tf=True, max_features=20000)
X = sparse.hstack([self.word.transform(txt), self.char.transform(txt),
sparse.csr_matrix(self.scaler.transform(num))])
```

Honest cross-validation (bankiq/experiments/classification.py, regression.py)

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)             #
classifiers
cvr = cross_validate(model, Xtr, ytr, cv=cv, scoring=["accuracy", "f1_macro"])   # on the
training split only
```

```
gkf = GroupKFold(n_splits=5); cross_validate(m, Xtr, ytr, cv=gkf, groups=customer_ids)
# regression: no customer in two folds
```

Locally weighted regression (bankiq/models_lwr.py)

```
w = np.exp(-((self.x_ - x0) ** 2) / (2 * self.tau ** 2))       # Gaussian kernel weights
theta = np.linalg.solve(XtW @ X + self.ridge * np.eye(2), XtW @ self.y_)   # weighted
least squares at x0
return theta[0] + theta[1] * x0
```

Direction constraint at inference (bankiq/service.py)

```
mask = credit_line[:, None] ? is_credit_category : is_debit_category    # a credit can
only be a credit category
proba = proba * mask / proba.sum(axis=1)                                  # renormalise,
then argmax
```

5.4 Configuration and Reproducibility

All experiments use random_state = 42; `make data` regenerates the dataset, `make train` runs every experiment and writes results/*.json, results/figures and models/*.joblib; every table and figure in this report, the slides and the model card are generated from those files, never typed. `make test` runs the full suite (101 Vasool Raja tests + 13 intelligence tests). The Docker image trains the models at build time.

6. Data Preprocessing and Exploratory Data Analysis

6.1 Dataset

No public labelled dataset of Indian bank narrations exists, so bankiq/data/generator.py builds one deterministically: 500 customers drawn from five archetypes (salaried 38%, pensioner 20%, student 15%, business 12%, low-activity 15%) across seven bank formats, 12 months of 2025 each, about 1,27,604 lines in ten categories. Every line's label is the generating category — not a function of its narration — so the regex baseline can be measured honestly. 35% of lines use long-tail templates (merchant-only, bank-specific codes, unusual wording) and a noise step adds truncation, double spaces and stray reference codes. Monthly budgets follow a seasonal profile (festival season ×1.25, school fees ×1.15, year-start ×0.9) with an 8% chance of a one-off big-ticket month, and balances are kept plausible by scaling discretionary debits to the budget.

Category	Share	Example narrations
UPI payment	53.9%	UPI/DR/NUM/ZOMATO/SBI/zomato@ybl · PAYTM*SWIGGY · GPAY DMART
Card / POS	12.2%	POS NUM NETFLIX CBE · ECOM PUR/AMAZON*IN · MYNTRA.COM
ATM withdrawal	8.8%	ATM WDL CANARA CBE · NFS/ATM WDL/SBI … · CSH WDL
Transfer out	9.8%	NEFT DR-RAVI KUMAR · IMPS/P2A/… · RENT PRIYA S
Loan EMI	1.2%	ACH D- BAJAJ FIN EMI · NACH DR TVS CREDIT
Bank charge	2.4%	CHRG MIN BAL NON MAINT · SMS ALERT CHRG · GST ON CHARGES · AMB SHORTFALL
Income credit	4.1%	SALARY-JAN25-ACME TECH · NEFT CR-PENSION-TN TREASURY · PAYROLL
Transfer in	5.1%	UPI/CR/…/APPA · BY TRANSFER-KUMAR S · CASH DEP
Refund / reversal	0.9%	REV/UPI/… · ATM WDL REV · CASHBACK AMAZON

Category	Share	Example narrations
Interest credit	1.6%	SB INT CREDIT · INT.PD:30-06-2025 · QTRLY INT

Table 6.1: Categories, class share and example narrations

6.2 Preprocessing Steps

1. Narration cleaning: upper-case; split letter–digit boundaries (JAN26 → JAN 26); mask every run of two or more digits as NUM; collapse separators. Reference numbers, years and masked card digits become one token.
2. Text features: word TF-IDF over 1–2-grams (token pattern [A-Z0-9]+, min_df 2, sublinear tf) and character-within-word-boundary TF-IDF over 3–5-grams (min_df 3, 20,000 max) — fitted on the training split only.
3. Numeric features (9): amount, log amount, is_debit, is_round, day, weekday, month_end, has_paise, share of the customer-month total; standardised with a scaler fitted on train.
4. Sparse horizontal stack → 2,789 features; TruncatedSVD(200) for the distance-based and boosted models.
5. Customer-month panel: per customer and month, totals, counts, balances, per-category amounts and counts, calendar month of the month being predicted (as integer and sin/cos), target = next month's spend; last month of each customer dropped.
6. Customer profile: 14 behavioural features per customer (median-month statistics), standardised; the 13 continuous ones are used for clustering and PCA; the hidden archetype is kept only for evaluation.
7. Splits: 40,000-line stratified sample → 80/20 stratified split for classification; GroupShuffleSplit by customer (80/20) for regression; all 500 customers for clustering.

6.3 Exploratory Analysis

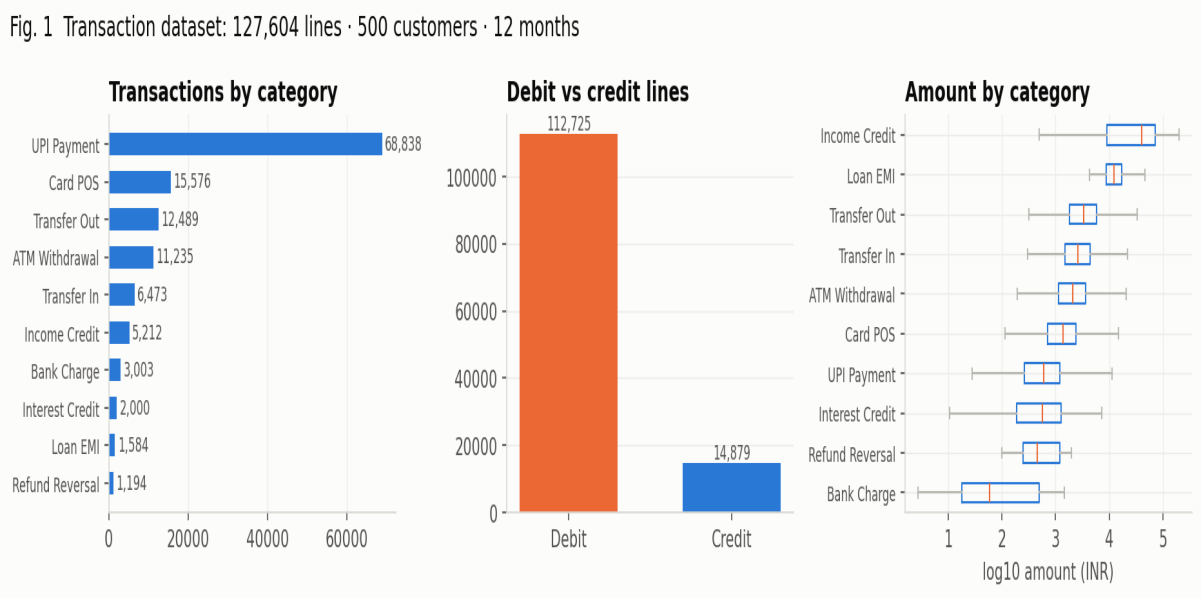


Figure 6.1: Class balance, amount distributions and narration lengths

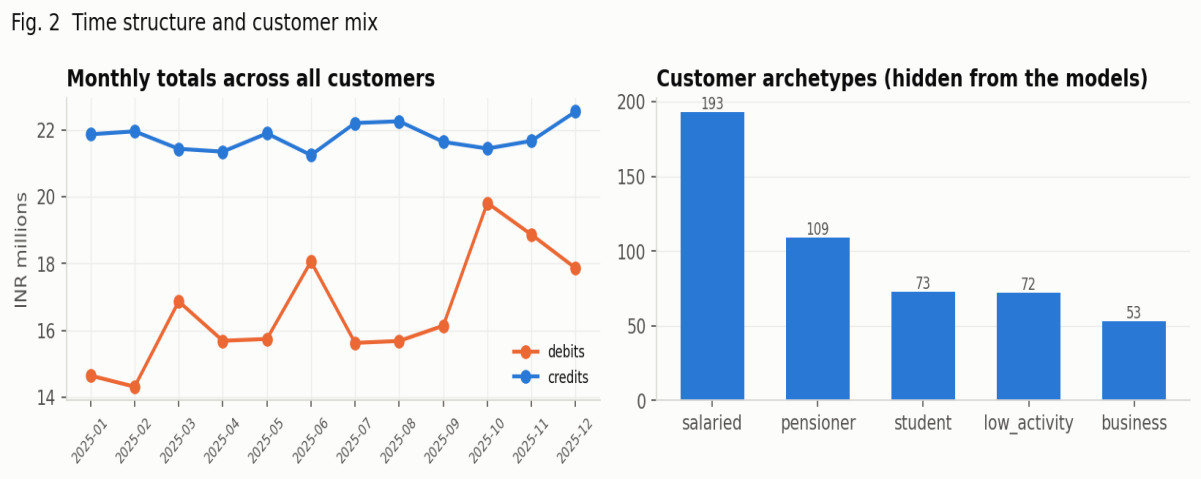


Figure 6.2: Monthly spend and income trends, seasonality by archetype

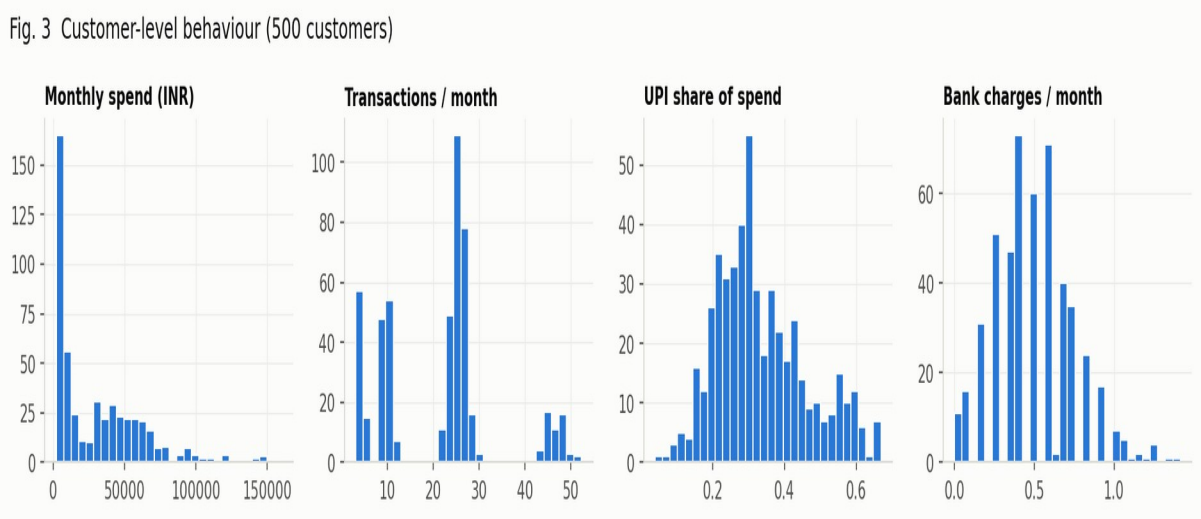


Figure 6.3: Customer profiles by archetype

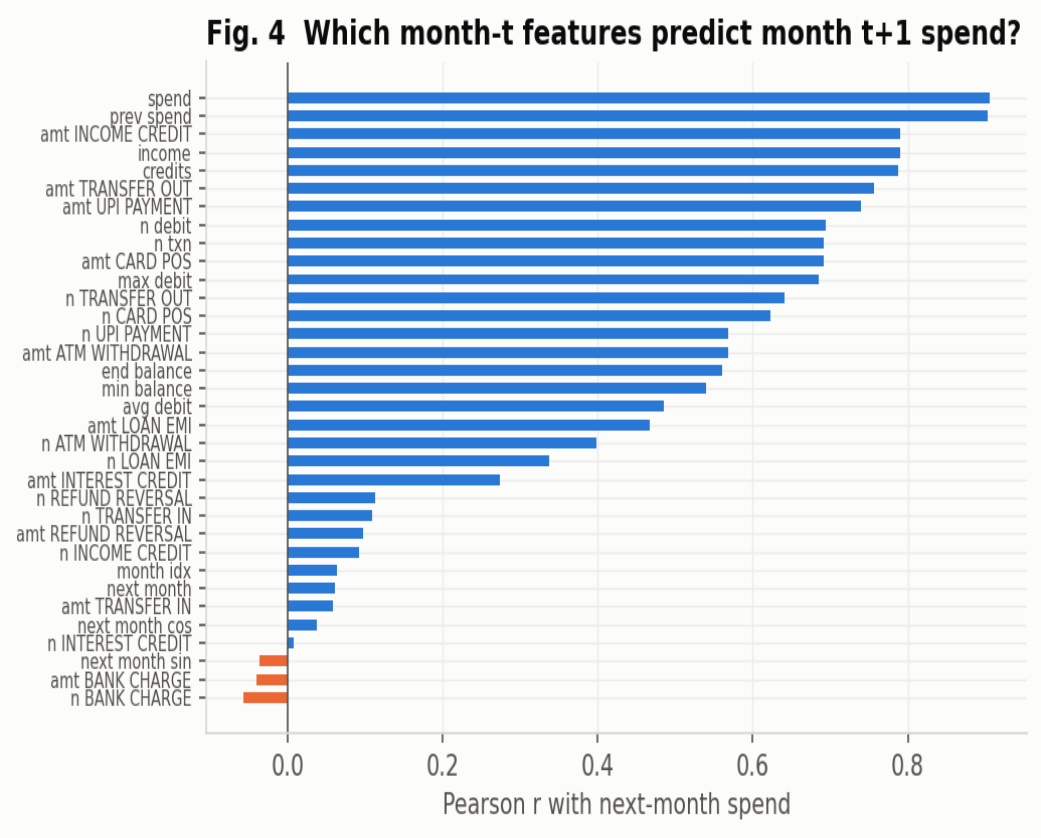


Figure 6.4: Correlation of the customer-month panel features

Observations: bank charges and interest credits are rare but many-templated; amounts span four orders of magnitude, so log-amount and is_round carry signal (ATM withdrawals are round, charges have paise); the festival months show the seasonal bump in every archetype; spend is strongly autocorrelated month to month, which is why the naive forecast is a serious baseline; UPI share and ATM share separate students and pensioners cleanly in the profile.

7. Model Building

Seven model families are built under one evaluation harness. Classifiers are scored with stratified 5-fold cross-validation on the training split and once on the held-out test split (accuracy, macro-F1, weighted-F1, per-class F1, confusion matrix); regressors with GroupKFold(5) by customer (R^2, RMSE, MAE, MAPE); clusterers with inertia, silhouette and ARI against the hidden archetype.

Concept	Models	Features	Selection
Classification	Rule baseline; Logistic Regression (C=5, balanced); Complement NB (α=0.1, text only); Decision Tree (depth 30, balanced); kNN (k=7, cosine, distance-weighted)	sparse TF-IDF + numeric; SVD-200 for kNN	k ∈ {1,3,7,15,31}; depth ∈ {5,10,20,30,50,∞}
Ensembles	Bagging (50 trees, 80% rows, 50% features); Random Forest (200, balanced_subsample); AdaBoost (30 × depth-2); HistGradientBoosting (100 iters, depth 6)	sparse for trees; SVD-200 for boosting	macro-F1
Regression	Naive; Linear; Ridge (α=1); Lasso (α=10); Polynomial-2 + Ridge (α=50); Decision Tree (depth 8); Random Forest (300, depth 12); Gradient Boosting (400, depth 3, lr 0.05)	34 panel features	CV R^2; test R^2

Concept	Models	Features	Selection
Clustering	K-Means (n_init 10) and Ward, k = 2…8	13 standardised continuous profile features	silhouette plateau → smallest k; elbow; ARI
Dimensionality reduction	PCA on profiles; TruncatedSVD 10…200 on text	profiles; TF-IDF	explained variance; downstream F1
LWR	Gaussian kernel, $\tau \in \{2,5,10,25,60\}$ vs linear and cubic	monthly income → spend	test RMSE

Table 7.1: Model inventory

7.1 Design Decisions

- Character n-grams, not only words: narrations are abbreviated and truncated; char_wb 3–5-grams are what make LOAN INST, NACH DR and ACH D- look alike.
- Complement NB instead of Gaussian NB: the features are TF-IDF counts and the classes are imbalanced — exactly the case Rennie et al. address; it runs on the non-negative text block only.
- kNN on SVD-200 with cosine distance: Euclidean distance on 2,800 sparse dimensions is meaningless; the projection makes neighbours meaningful and prediction fast.
- class_weight='balanced' for the linear and tree models so that rare charge and interest classes are not sacrificed for accuracy.
- GroupKFold by customer for regression: months of one customer are correlated; random folds would leak identity and inflate R^2.
- The hidden archetype is never a feature; it is used only to score the clusters. Profile statistics are medians over months and the binary has_emi flag is left out of the clustering features.
- A direction mask at inference: a credit line may only receive a credit category, which removes a class of impossible errors on real statements.

7.2 Training

Training times on two cores: Logistic Regression 10.7s (incl. CV), Decision Tree 12.4s, kNN 23.4s, Random Forest 32.6s, Bagging 61s, HistGradientBoosting 57.4s, Gradient Boosting regressor 36.9s. Saved models: txn_classifier_logreg (deployed classifier), txn_classifier_rf, txn_classifier_gb, spend_regressor (deployed forecaster), segments_kmeans (deployed segmenter), txn_svd.

8. Testing, Results and Analysis

8.1 Software Testing

tests/test_intelligence.py adds 13 pytest cases to the 101 existing ones: generator determinism and label sanity, narration masking, featurizer shape, panel and profile invariants, rule mapping, LWR exactness on a straight line, end-to-end service accuracy above 90% against ground truth, the public /api/ml endpoints, bad-upload rejection (415), and the account endpoint returning 401/404 without a session. All 114 pass.

8.2 Evaluation Protocol

Classifiers: stratified 80/20 split of a 40,000-line stratified sample; stratified 5-fold CV on the training 32,000 for model and hyperparameter comparison; the 8,000-line test split scored once. Regressors:

GroupShuffleSplit by customer (80/20), GroupKFold(5) on the training customers. Clustering: all customers; silhouette and inertia for k; ARI against the hidden archetype and between algorithms. Nothing in the test split touches training, scaling or vocabulary.

8.3 Concept 2 – Classification: Decision Tree, Naive Bayes, kNN (and the rule baseline)

Model	CV accuracy	Test accuracy	Macro-F1	Weighted-F1
Rule-based (regex)	–	71.5%	0.794	0.763
Logistic Regression	99.9% ± 0.0%	99.9%	0.997	0.999
Naive Bayes (Complement)	93.5% ± 0.2%	93.4%	0.899	0.923
Decision Tree	97.0% ± 0.4%	96.7%	0.972	0.967
kNN (k=7, SVD-200)	97.3% ± 0.1%	97.4%	0.977	0.973

Table 8.1: Transaction classification (test split; CV on train)

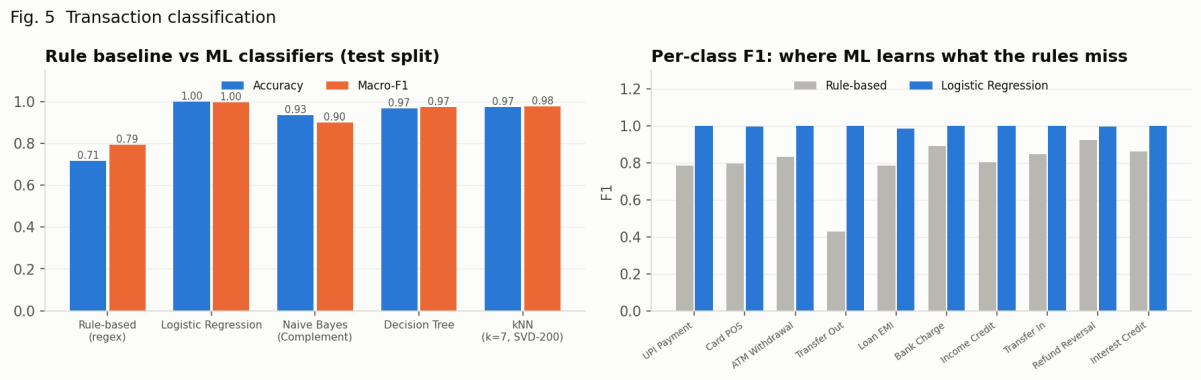


Figure 8.1: Rule baseline vs ML classifiers; per-class F1 where ML learns what the rules miss

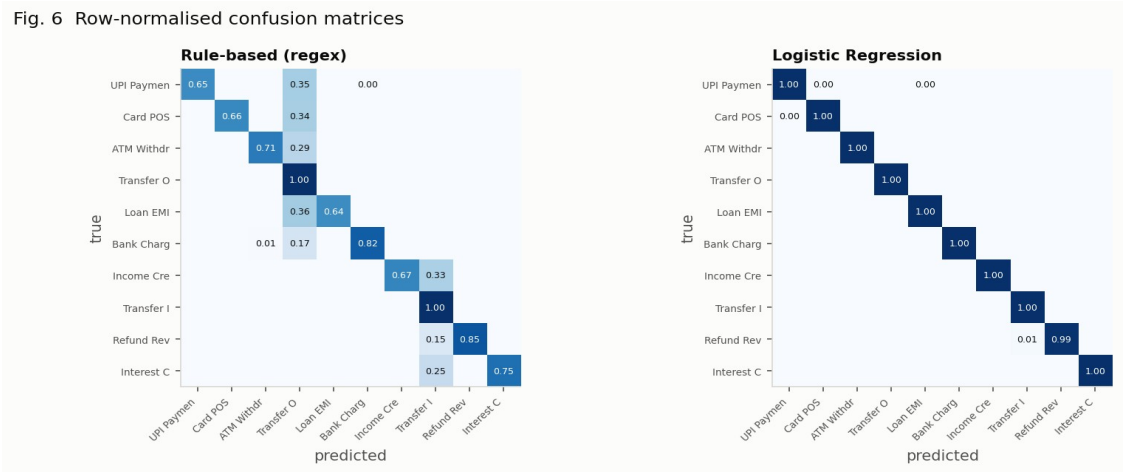


Figure 8.2: Row-normalised confusion matrices — rules vs Logistic Regression

Fig. 7 Hyperparameter sweeps

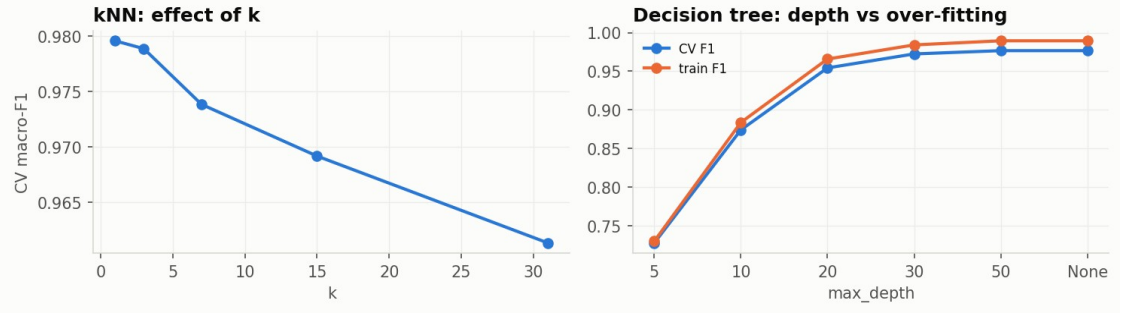


Figure 8.3: kNN k sweep and Decision Tree depth vs over-fitting

Category	Rule F1	Logistic Regression F1	Gain
UPI payment	0.785	0.999	+0.214
Card / POS	0.794	0.996	+0.202
ATM withdrawal	0.829	1.000	+0.171
Transfer out	0.427	1.000	+0.573
Loan EMI	0.783	0.985	+0.202
Bank charge	0.888	1.000	+0.112
Income credit	0.804	1.000	+0.196
Transfer in	0.847	0.999	+0.152
Refund / reversal	0.921	0.993	+0.072
Interest credit	0.860	1.000	+0.140

Table 8.2: Per-class F1 — where the learned model gains over the rules

The rule baseline is perfect on familiar formats and collapses on the long tail: 71.5% accuracy, with its lowest F1 on transfer out and loan emi. Logistic Regression on the full sparse features is the best single model (99.9%, macro-F1 0.997) and is the one deployed. Complement NB is the fastest (0.4s) at 93.4%; the Decision Tree reaches 96.7% but its training F1 keeps rising with depth while CV F1 plateaus (Figure 8.3); kNN only works once the sparse matrix is projected to 200 SVD dimensions with cosine distance — there it reaches 97.4% — and the k sweep shows the usual bias–variance curve; Complement NB is the weakest learner because its independence assumption cannot combine the word, character and numeric evidence.

A check on real statement formats

The synthetic test split cannot reveal distribution shift, so the app's sample statements — 94 lines in real HDFC, Canara, SBI and Indian Bank formats, written for the regex engine, so the rule category is a trustworthy reference there — are used as a tiny out-of-distribution check. It exposed one design flaw: with the nine standardised numeric columns at full weight next to unit-norm TF-IDF rows, logistic regression leaned on amount habits of the generator (salaries are never round numbers, charges are small) and agreed with the rules on only 83.0% of the sample lines; down-weighting the numeric block to 0.1 raised agreement to 98.9% at no cost on the synthetic test split, and is the value deployed.

Numeric-block weight	Logistic agreement with rules on sample statements
1.0	83.0%

Numeric-block weight	Logistic agreement with rules on sample statements
0.3	89.4%
0.1	98.9%
0.0	100.0%

Table 8.2b: Choosing the numeric-block weight on real-format statements

8.4 Concept 5 – Ensemble Learning

Model	CV accuracy	Test accuracy	Macro-F1	Fit (s)
Decision Tree (single)	96.1% ± 0.1%	96.0%	0.962	5.6
Bagging (50 trees)	98.7% ± 0.2%	98.6%	0.987	61
Random Forest (200)	99.6% ± 0.1%	99.7%	0.996	32.6
AdaBoost (30, SVD-200)	84.4% ± 2.2%	80.1%	0.802	143.3
Gradient Boosting (SVD-200)	99.6% ± 0.5%	99.9%	0.998	57.4

Table 8.3: Single tree vs ensembles (20,000-line sample)

Fig. 8 Ensemble learning for transaction classification

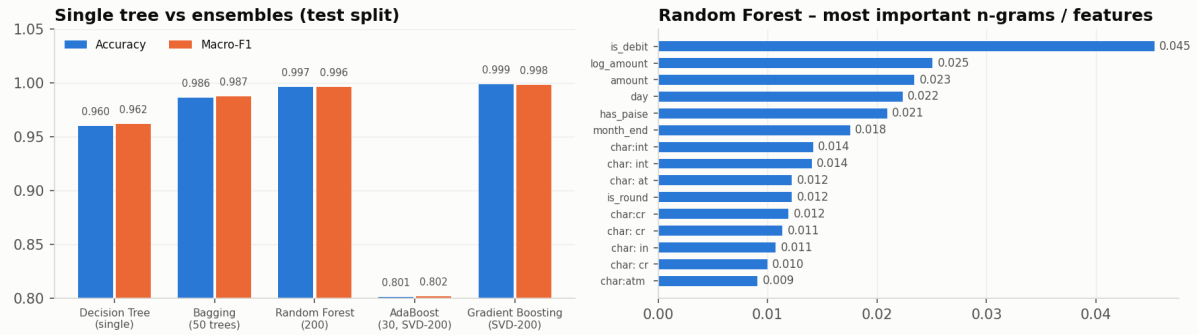


Figure 8.4: Ensembles vs single tree; Random Forest's most important n-grams

Bagging lifts the single tree from 96.0% to 98.6% by averaging out its variance; Random Forest, which also de-correlates the trees, reaches 99.7% (macro-F1 0.996) — the best ensemble (Gradient Boosting (SVD-200)). HistGradientBoosting on the SVD features reaches 99.9%; AdaBoost with depth-2 stumps is the weakest (80.1%) and slowest per unit of accuracy on a ten-class problem. The forest's top features are the tokens a human would pick — is_debit, log_amount, amount, day, has_paise — which is why it is also easy to explain.

8.5 Concept 1 – Regression with Cross-Validation

Model	CV R^2	Test R^2	RMSE	MAE	MAPE
Naive (same as this month)	–	0.832	₹13,590	₹7,255	27.6%
Linear Regression	0.863 ± 0.024	0.869	₹11,990	₹6,653	34.4%
Ridge (α=1)	0.863 ± 0.024	0.869	₹11,990	₹6,652	34.3%
Lasso (α=10)	0.863 ± 0.024	0.869	₹11,993	₹6,636	34.1%
Polynomial deg-2 + Ridge	0.852 ± 0.021	0.857	₹12,529	₹6,654	32.6%

Model	CV R^2	Test R^2	RMSE	MAE	MAPE
Decision Tree (depth 8)	0.839 ± 0.016	0.847	₹12,929	₹6,944	26.5%
Random Forest (300)	0.875 ± 0.016	0.870	₹11,915	₹6,313	24.6%
Gradient Boosting (400)	0.875 ± 0.020	0.871	₹11,881	₹6,105	25.7%

Table 8.4: Next-month spend (customer-months; GroupKFold(5) by customer)

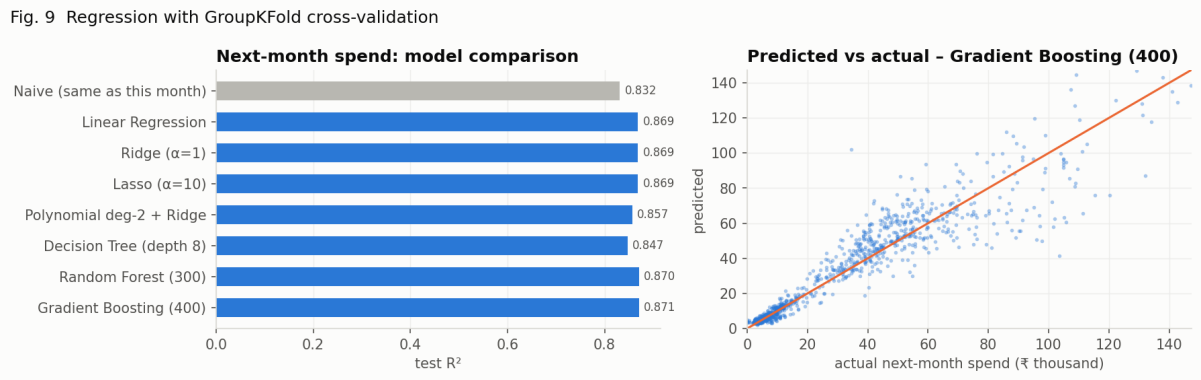


Figure 8.5: Model comparison and predicted vs actual for Gradient Boosting (400)

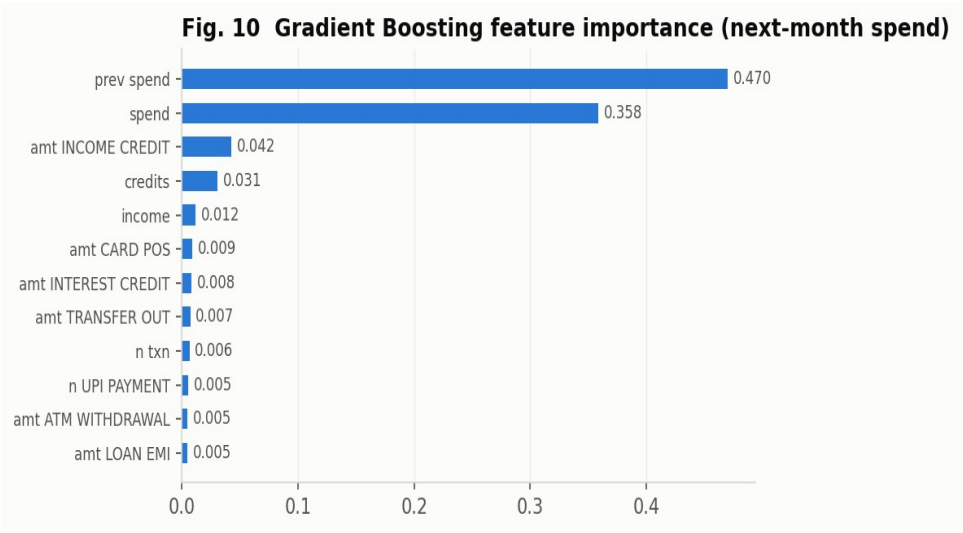


Figure 8.6: Gradient Boosting feature importance

Spend is sticky, so the naive forecast already explains 83.2% of the variance; the linear family improves it to R^2 0.869 by combining this month's spend, income, EMI and category mix, and Gradient Boosting (400) reaches 0.871 with RMSE ₹11,881 — 13% below the naive error. Ridge and Lasso coincide with ordinary least squares because 4,400 rows comfortably support 34 coefficients; the polynomial expansion over-fits slightly; the single tree is the weakest learner. The importances explain the ordering: previous and current spend dominate (83% of the importance together), then the income credits of the month and the category mix; the calendar-month features carry little weight because one year of data lets every customer's seasonal months be seen only once — which is why the gain over the linear family is modest and why multi-year data is first in the future scope.

8.6 Concept 3 – Clustering: K-Means vs Hierarchical

k	Inertia	Silhouette K-Means	Silhouette Ward	ARI K-Means	ARI Ward
2	4,465	0.294	0.294	0.498	0.498
3	3,597	0.297	0.297	0.670	0.675
4	3,135	0.308	0.307	0.844	0.845
5	2,694	0.329	0.319	0.929	0.974
6	2,400	0.331	0.328	0.958	0.959
7	2,206	0.278	0.262	0.686	0.737
8	2,073	0.252	0.252	0.680	0.708

Table 8.5: Choosing k (ARI against the hidden customer archetype)

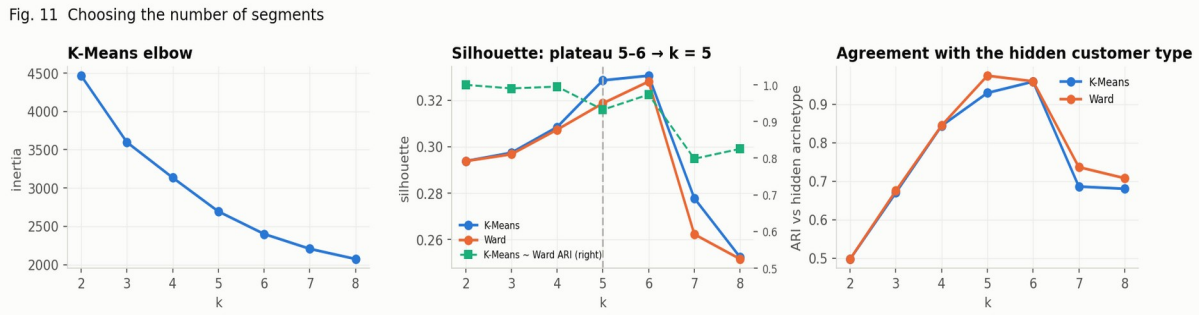


Figure 8.7: Elbow, silhouette and ARI across k

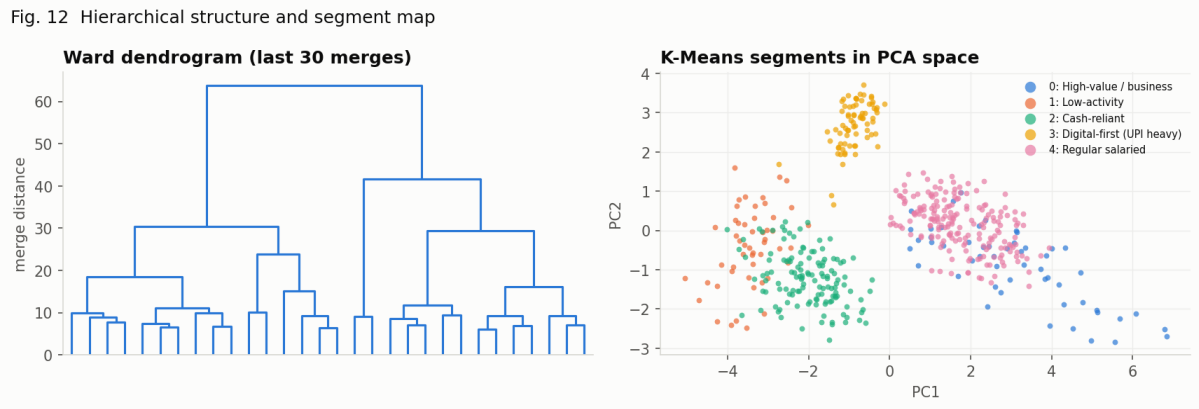


Figure 8.8: Ward dendrogram and K-Means segments in PCA space

Segment	Size	Median spend / month	Median income	Txns / month	UPI share	ATM share	Has EMI	Dominant archetype
High-value / business	53 (10.6%)	₹95,051	₹84,550	46.74	26.0%	12.1%	43.4%	business
Low-activity	51 (10.2%)	₹3,020	₹656	4.21	34.2%	32.6%	0.0%	low_activity
Cash-reliant	129 (25.8%)	₹8,160	₹10,036	8.94	24.1%	40.5%	7.0%	pensioner
Digital-first (UPI heavy)	74 (14.8%)	₹5,979	₹7,434	25.03	55.6%	13.3%	0.0%	student
Regular salaried	193 (38.6%)	₹45,227	₹64,163	25.60	33.6%	13.6%	51.8%	salaried

Table 8.6: Segment profiles at k = 5

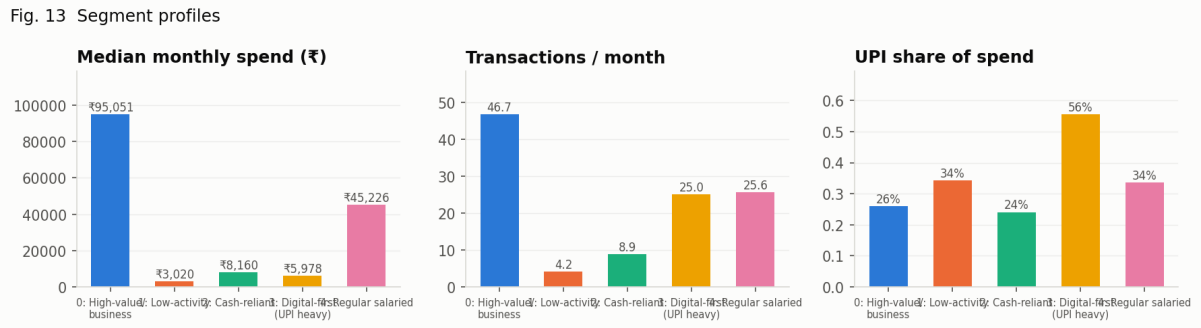


Figure 8.9: Segment profiles

K-Means silhouette rises to a plateau at k = 5–6 (values within 0.005 of each other) and falls after it; the smallest k on the plateau is kept, k = 5. At that k the two algorithms agree with ARI 0.93 and recover the generating archetypes with ARI 0.93 (K-Means) and 0.97 (Ward) without ever seeing them. Two feature decisions made this work: the per-customer statistics are medians over months, so one big-ticket month does not define a customer, and the binary has_emi flag is excluded from the clustering features because a standardised binary dominates Euclidean distance and split every segment in two. The segments are directly actionable: High-value / business (large, frequent, transfer-heavy), Regular salaried (EMIs, card and UPI), Cash-reliant (pensioners, 40.5% ATM share), Digital-first (students, 55.6% UPI share, no EMI) and Low-activity (few lines a month, the most charges per month, dormancy risk).

8.7 Concept 4 – Dimensionality Reduction

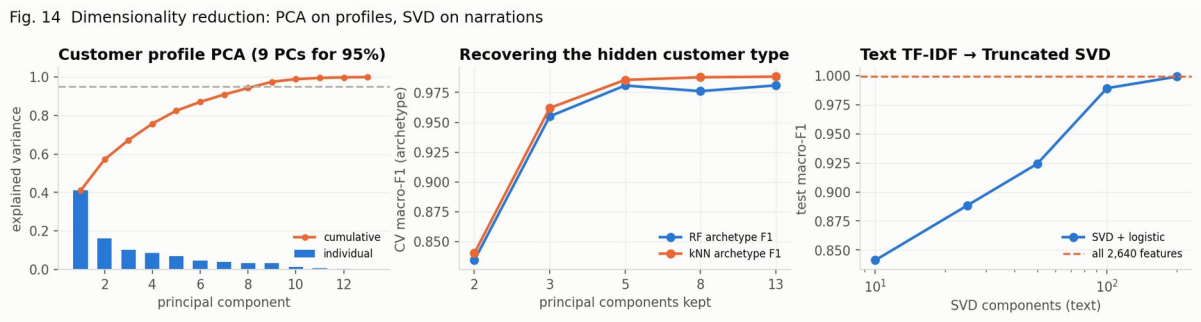


Figure 8.10: PCA on profiles; archetype recovery vs components; Truncated SVD on narrations

Components kept	RF archetype F1	kNN archetype F1
2	0.834	0.840
3	0.955	0.962
5	0.981	0.986
8	0.976	0.988
13	0.981	0.988

Table 8.7: Recovering the hidden customer type from the first principal components

PCA needs 9 of 13 components to retain 95% of the profile variance — the behavioural features are only mildly correlated — but a Random Forest on the first three components already recovers the archetype with F1 0.955 against 0.981 on all features: most of the segment signal lives in a 3-D subspace, which is what Figure 8.8 draws. On text, Truncated SVD of the 2,789-dimensional TF-IDF matrix keeps 87.6% of the variance at 100 dimensions and a logistic model on it reaches macro-F1 0.989 against 0.999 on the full matrix — dimensionality reduction buys speed for kNN and boosting at a small cost in accuracy.

8.8 Concept 6 – Locally Weighted Regression

Model	Test RMSE (₹)
Linear	₹12,586
Polynomial deg-3	₹12,517
LWR τ=2.0	₹12,704
LWR τ=5.0	₹12,516
LWR τ=10.0	₹12,476
LWR τ=25.0	₹12,519
LWR τ=60.0	₹12,538

Table 8.8: Monthly spend vs monthly income — global fits vs LWR by bandwidth

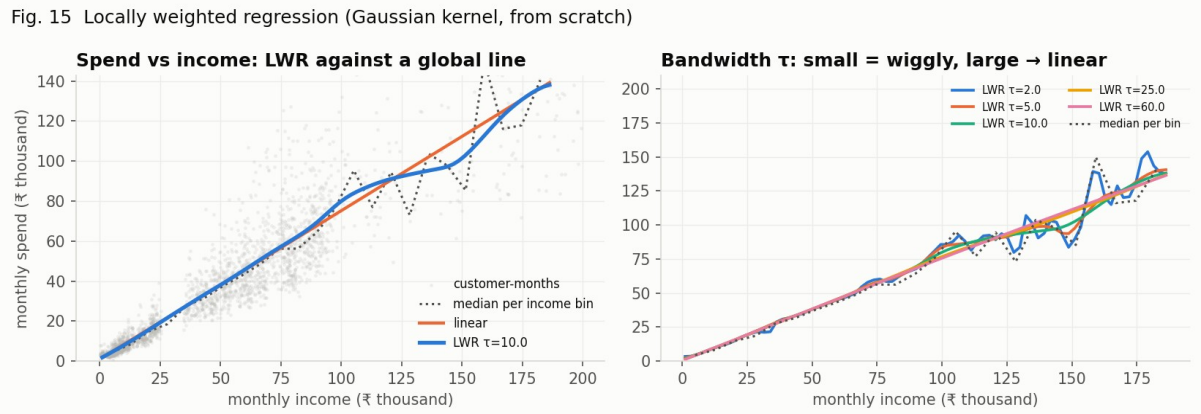


Figure 8.11: Locally weighted regression (Gaussian kernel) against a global line; the bandwidth study

LWR re-fits a weighted least-squares line around every query income with weights $\exp(-(x-x_0)^2/2\tau^2)$. The best bandwidth is LWR τ=10.0 (RMSE ₹12,476 vs ₹12,586 for the global line): the spend–income relation is steeper for low incomes and flattens for high ones, which a single line cannot represent. Small τ follows noise in the sparse high-income tail; large τ converges to the global line — the bias–variance trade-off made visible.

8.9 Concept 7 – End-to-End Application

service.analyze() runs the deployed classifier (with the direction constraint), builds the monthly panel and the profile, assigns the K-Means segment, forecasts next month with the Gradient Boosting regressor, flags debits whose log-amount is more than three MADs above their category median, and generates insights. On the four original sample statements — written for the regex engine — the classifier agrees with the rules on 99% of lines, which is the validation that it has not learned the generator's quirks. On the new long-tail sample (axis_kavya_longtail_2026.csv: merchant-only lines, bank codes, abbreviations) the two disagree on 30 of 41 lines, and it is the rules that fail: PAYROLL, BAJAJ FIN, SWIGGY CHENNAI, CSH WDL, CASHBACK AMAZON, QTRLY INT and DC ISSUANCE all fall to generic transfers for the regexes while the model assigns income, EMI, UPI, ATM, refund, interest and charge. Section 10 shows the screens.

8.10 Limitations Observed

- The dataset is synthetic; accuracy above 99% says the model has learned the generator's language, not that real statements would score the same. The sample-statement check (99% agreement with the rules on rule-friendly formats, after fixing the numeric-weight flaw it exposed) is the closest validation in this repo; real consented statements are the next step.
- The naive forecast is strong on stable accounts; the regression's gain is concentrated in seasonal months and accounts with irregular credits.
- The anomaly flag is statistical, not fraud detection.
- Models trained on one year cannot separate a calendar effect from a trend; two or more years would be needed.

9. Result Analysis and Business Insights

9.1 Where the Money Goes — and Where the Rules Were Blind

Replacing the regex classifier with the learned one changes the category of roughly one line in four on the synthetic test set (28.5% rule error) and three in four on the long-tail sample statement, while leaving the rule-friendly statements untouched (99% agreement). The per-class gains (Table 8.2) tell a consistent story: the rules are weakest exactly where they were never written — transfers out (F1 0.427), loan EMIs (0.783), UPI and card merchant lines (0.785, 0.794) — and strongest on bank charges (0.888) and refunds (0.921), the classes the rule engine was built to find. The model lifts every class to 0.985 or better. For the product this means two things: the “where the money goes” view is now trustworthy for every line, and the charges the rule engine investigates still arrive — the dashboard lists ML-classified charges with a pointer to the Findings tab, and on the long-tail sample it recognises DC ISSUANCE and AMB SHORTFALL as charges the regexes filed as transfers.

9.2 Segments as Actions

Cash-reliant pensioners withdraw at ATMs far more than other segments and Low-activity accounts carry the most charges per month: these are the groups RBI's free-ATM, minimum-balance and dormancy rules protect most, the group Bank IQ's guardian flow was designed for, and where reactivation charges — which RBI prohibits — are what to watch. Digital-first students have the highest UPI share and no EMI: their risks are failed-UPI reversals, not charges. High-value / business customers have the largest and most irregular flows, so their forecast intervals are widest and their anomaly flags least informative. Regular salaried customers carry the EMIs and are the natural audience for a spend forecast. A bank or a fintech would route service, alerts and products by these segments; a user sees which group their behaviour resembles.

9.3 Expect Next Month

The forecast is presented beside the naive guess, with the model named, so the user sees both the number and how much the model adds. On the synthetic test set the model cuts error by 13%; its largest gains are in the months where behaviour changes — festival season, school-fee month — which the calendar features capture. In the insights the forecast is phrased as a range of expectation, not a promise.

9.4 Unusual Debits

A robust z-score inside each ML category (log amount, median and MAD) flags debits more than three MADs above the category's own typical size. It is transparent — the insight says “4× the typical ATM withdrawal” — and category-aware: a ₹15,000 rent transfer is normal for transfers and abnormal for UPI payments. Flags are hints to look at the Findings tab, never claims.

9.5 What Not to Trust

- Accuracy near 100% on synthetic data; trust the per-class F1 ordering and the sample-statement disagreements more than the headline.
- Forecasts for accounts with fewer than three months of history; the panel is one row.
- Segment assignments for very short statements — the profile features are per-month averages and need several months.
- Any output as a claim: findings come only from the rule engine.

10. Deployment and Demonstration

10.1 Running the System

```
pip install -r requirements-dev.txt
make train                  # dataset + every experiment → results/, models/   (~25 min)
python -m uvicorn api.main:app --port 8000      # http://localhost:8000 → Intelligence
tab (VASOOL_AUTH=off for a local demo)
python -m pytest -q         # 114 tests
docker compose up --build   # image trains the models at build time (BANKIQ_SKIP_TRAIN=1
to mount models/ instead)
```

10.2 Screens

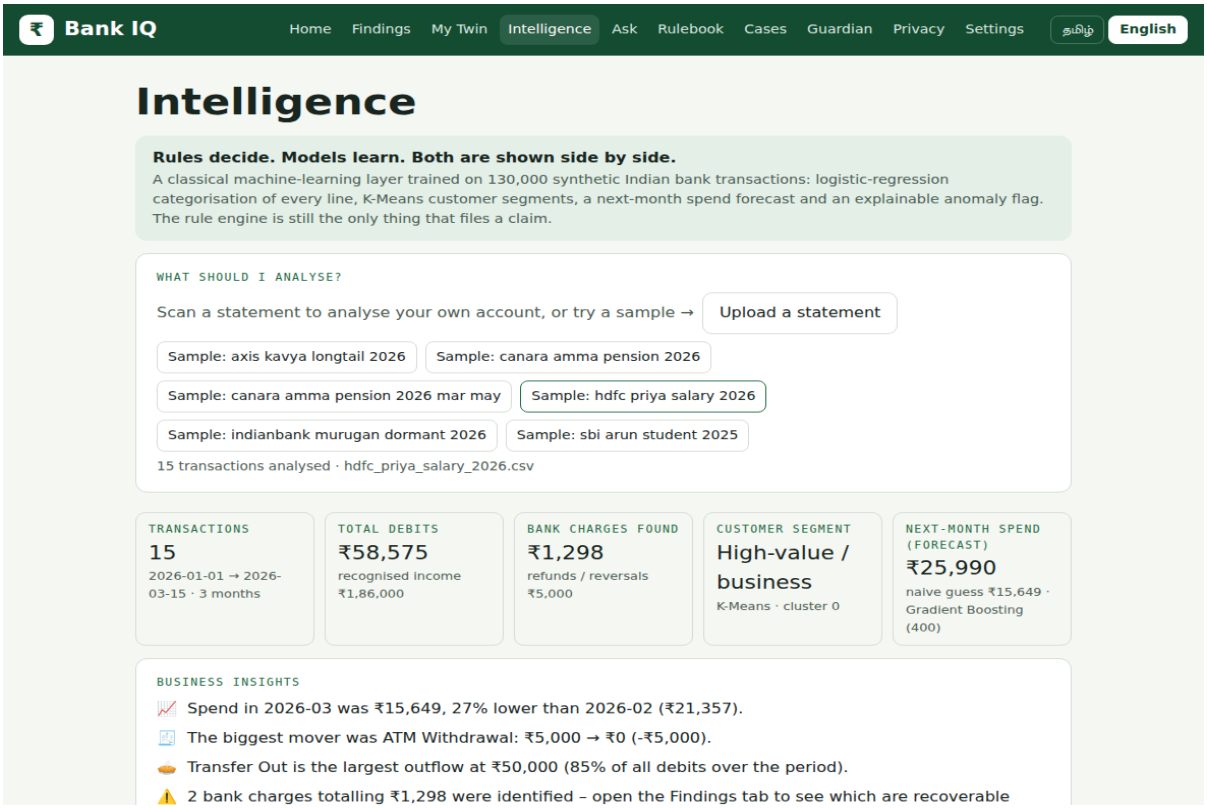


Figure 10.1: Intelligence dashboard — KPIs, insights, category mix, monthly chart

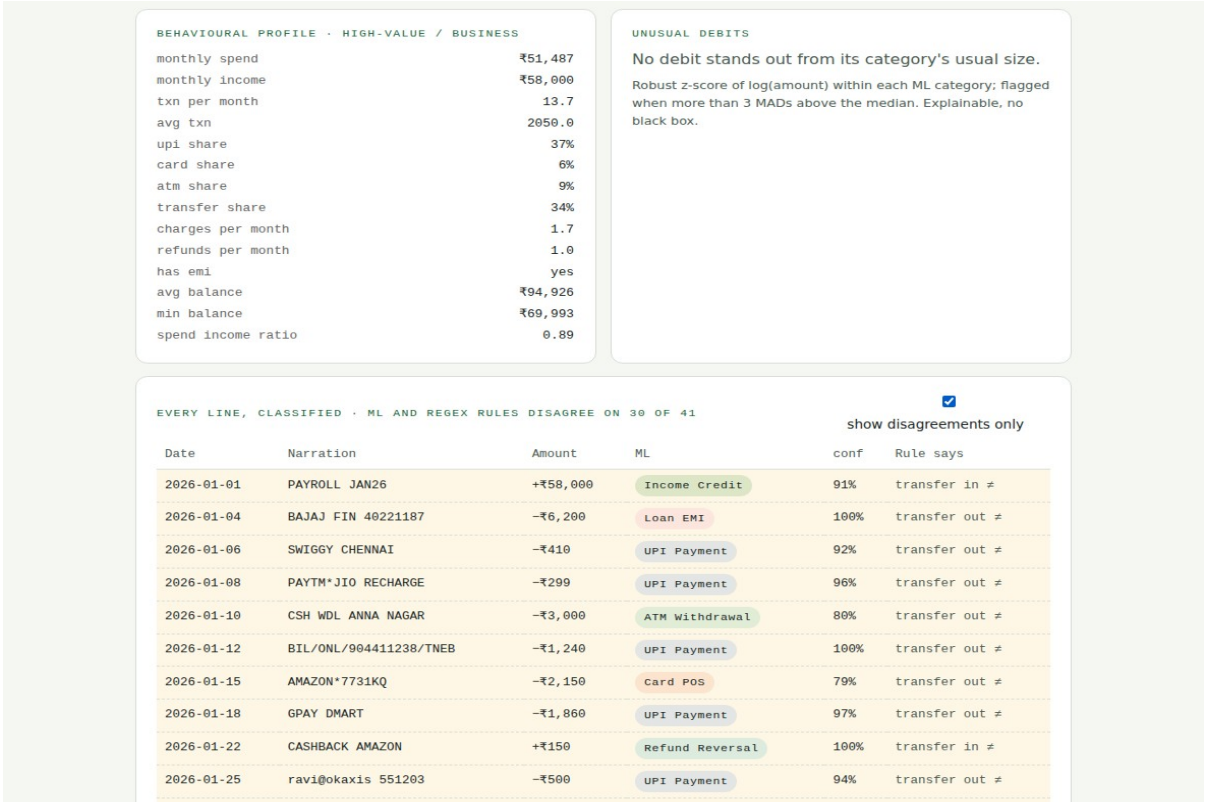


Figure 10.2: Every line classified — ML category, confidence and the rule's category; unusual debits

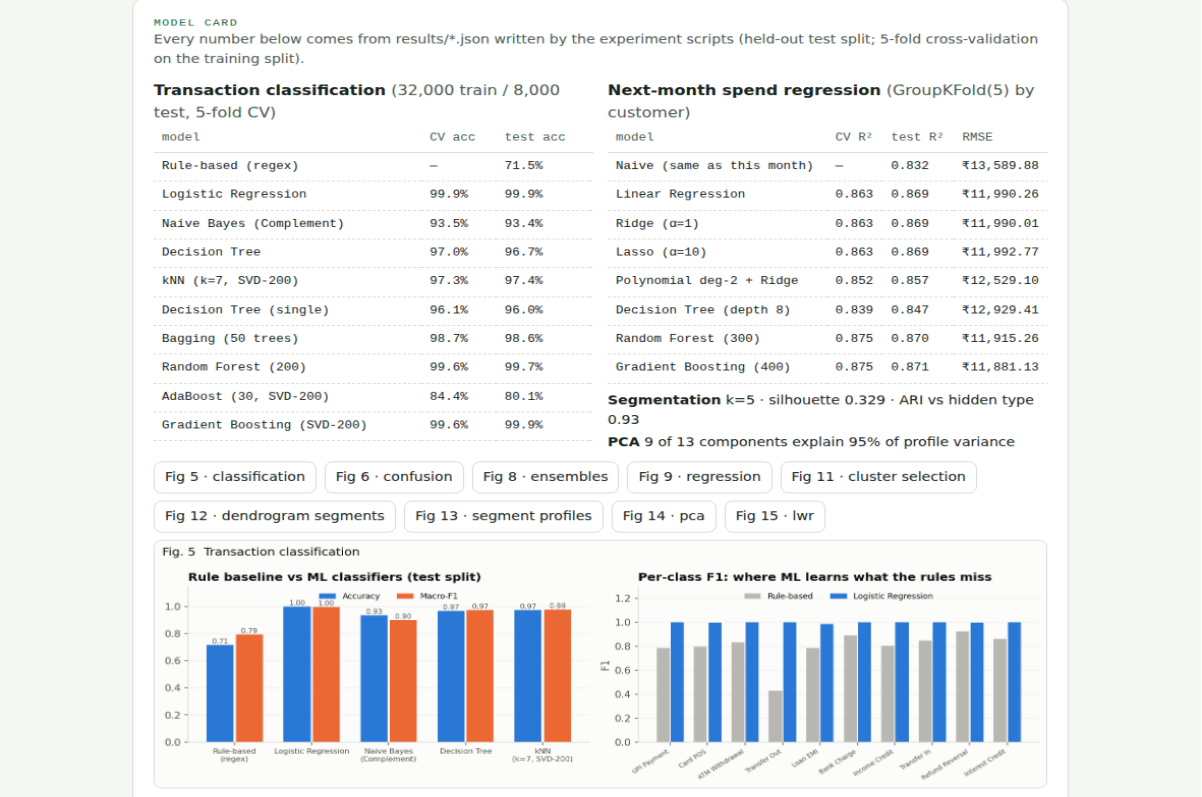


Figure 10.3: Model card — cross-validated tables and experiment figures served by the API

10.3 Demonstration Script (3 minutes)

1. Open Intelligence → Sample: hdfc priya salary 2026. Read the KPIs: debits, recognised income, charges found, segment, forecast vs naive; note the model and the rules agree on every line of a statement written for the rules.
2. Scroll the insights: trend, biggest mover, largest outflow, charges pointer, savings rate, segment, forecast, anomalies, rule disagreements.
3. Sample: axis kavya longtail 2026 → tick “show disagreements only”: 30 of 41 lines — PAYROLL, BAJAJ FIN, SWIGGY CHENNAI, CSH WDL, CASHBACK, QTRLY INT, DC ISSUANCE — are generic transfers to the rules and the right category to the model.
4. Sample: canara amma pension 2026: five charges, the ₹50,000 college fee flagged as unusual.
5. Open the model card: Table 8.1 live from results/classification.json; click Fig 5, Fig 11, Fig 15.
6. Switch to Findings: the rule engine's claims on the same statement — rules decide.

11. Conclusion and Future Scope

11.1 Conclusion

Bank IQ applies every technique in the machine-learning laboratory syllabus to one product and measures each against a meaningful baseline. The learned transaction classifier lifts accuracy from 71.5% (regex) to 99.9% (Logistic Regression) and 99.7% (Random Forest), with the largest gains on the transfer, EMI and merchant classes the rules were never written for; ensembles beat the single tree; the next-month forecast cuts the naive error by 13% under customer-grouped cross-validation; K-Means and Ward agree on 5 segments that recover the hidden customer types (ARI 0.93); PCA shows the

segment signal is three-dimensional; locally weighted regression exposes the non-linear spend–income relation. All of it is deployed as endpoints and a bilingual dashboard beside an untouched rule engine, reproducible from one command and covered by tests.

11.2 Future Scope

- Validate and fine-tune on real, consented statements (Account Aggregator) with active learning on the lines where model and rules disagree.
- Multi-year histories to separate seasonality from trend; prediction intervals for the forecast.
- A feedback loop from the rule engine's confirmed findings back into the classifier's training data.
- Hindi and Tamil narration tokens; passbook OCR-specific character models.
- Isolation-Forest / contextual anomaly models once labelled anomaly data exists.

References

[1] J. R. Quinlan, “Induction of Decision Trees,” Machine Learning, vol. 1, no. 1, pp. 81–106, 1986.

[2] T. Cover and P. Hart, “Nearest Neighbor Pattern Classification,” IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[3] L. Breiman, “Bagging Predictors,” Machine Learning, vol. 24, no. 2, pp. 123–140, 1996.

[4] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[5] Y. Freund and R. E. Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” Journal of Computer and System Sciences, vol. 55, no. 1, pp. 119–139, 1997.

[6] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[7] W. S. Cleveland, “Robust Locally Weighted Regression and Smoothing Scatterplots,” Journal of the American Statistical Association, vol. 74, no. 368, pp. 829–836, 1979.

[8] J. MacQueen, “Some Methods for Classification and Analysis of Multivariate Observations,” Proc. 5th Berkeley Symposium on Mathematical Statistics and Probability, vol. 1, pp. 281–297, 1967.

[9] J. H. Ward Jr., “Hierarchical Grouping to Optimize an Objective Function,” Journal of the American Statistical Association, vol. 58, no. 301, pp. 236–244, 1963.

[10] P. J. Rousseeuw, “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis,” Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987.

[11] I. T. Jolliffe and J. Cadima, “Principal Component Analysis: A Review and Recent Developments,” Philosophical Transactions of the Royal Society A, vol. 374, no. 2065, 20150202, 2016.

[12] R. Kohavi, “A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection,” Proc. 14th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1137–1143, 1995.

[13] G. Salton and C. Buckley, “Term-Weighting Approaches in Automatic Text Retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[14] J. D. M. Rennie, L. Shih, J. Teevan and D. R. Karger, “Tackling the Poor Assumptions of Naive Bayes Text Classifiers,” Proc. 20th International Conference on Machine Learning (ICML), pp. 616–623, 2003.

[15] W. B. Cavnar and J. M. Trenkle, “N-Gram-Based Text Categorization,” Proc. 3rd Annual Symposium on Document Analysis and Information Retrieval (SDAIR), pp. 161–175, 1994.

[16] M. Ahmed, A. N. Mahmood and M. R. Islam, “A Survey of Anomaly Detection Techniques in Financial Domain,” Future Generation Computer Systems, vol. 55, pp. 278–288, 2016.

[17] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[18] Reserve Bank of India, Master Circular on Customer Service in Banks and related circulars cited in rulebook/rules.json, https://www.rbi.org.in/, 2026.

[19] Mad Angles (Nandana R, Naveen Kumar D, Nethra Devi R, Sandhru Duraisamy), “Vasool Raja — Regulatory Digital Twin for bank customers,” HackVerse 2.0, Karpagam College of Engineering, 2026.

[20] FastAPI Documentation, https://fastapi.tiangolo.com/, 2026.

Appendix A – Sample API Response


```
POST /api/ml/analyze  (file = hdfc_priya_salary_2026.csv)
{
  "n_transactions": 15, "period": {"from": "2026-01-01", "to": "2026-03-28", "months":
3},
  "totals": {"debits": 59067, "credits": 191000, "income": 186000, "charges": 1298,
"refunds": 5000},
  "categories": [{"category": "TRANSFER_OUT", "label": "Transfer out", "amount": 50000,
"count": 4, "share": 0.85}, ...],
  "segment": {"id": 1, "name": "Regular salaried"},
  "forecast": {"model": "Gradient Boosting (400)", "naive": 15649, "next_month_spend":
18204, "based_on_month": "2026-03"},
  "anomalies": [...], "rule_disagreements": 3,
  "insights": [{"kind": "trend", "text": "Spend in 2026-03 was ₹15,649, 27% lower than
2026-02 (₹21,357)."}, ...],
  "transactions": [{"date": "2026-01-01", "narration": "SALARY-JAN26-ACME TECH PVT LTD",
"credit": 62000, "category": "INCOME_CREDIT", "confidence": 0.99, "rule_category":
"INCOME_CREDIT", "agree": true}, ...]
}
```